

10.3 Householder and Givens QR

Orthogonal matrices preserve lengths (Proposition 1.5.3(a)). For any orthogonal Q ,

$$\|y - Xb\| = \|Q^T y - Q^T Xb\| \quad \text{for every } b.$$

So a least squares problem can be replaced by an equivalent one in which $Q^T X$ is upper triangular, without ever forming $X^T X$. Such a Q is built from reflections, which zero a whole column below the diagonal at once, or from rotations, which zero one entry at a time.

10.3.1 Householder reflections

A reflection $H = I - 2vv^T/(v^T v)$ is symmetric and orthogonal, and it maps v to $-v$ and fixes every vector orthogonal to v (Exercise (B1)). The whole numerical content lies in choosing v so that a given vector is mapped onto a coordinate axis, and in choosing its sign so that no cancellation occurs.

Lemma 10.3.1 — Householder vector

Let $x \in \mathbb{R}^m$, $x \neq 0$, let $\alpha = -\text{sign}(x_1)\|x\|$ (with $\text{sign}(0) = 1$), and let $v = x - \alpha e_1$. Then $v \neq 0$, and $H = I - 2vv^T/(v^T v)$ satisfies $Hx = \alpha e_1$. The first entry $v_1 = x_1 - \alpha$ is a sum of two numbers of the same sign, so $|v_1| = |x_1| + \|x\|$. Applying H to a vector w as $w - 2v(v^T w)/(v^T v)$ costs about $4m$ flops, and H is never formed.

Proof. α and x_1 have opposite signs (or $x_1 = 0$), so $v_1 = x_1 - \alpha$ adds two numbers of the same sign and $|v_1| = |x_1| + \|x\| > 0$. Next, $v^T x = \|x\|^2 - \alpha x_1$ and $v^T v = \|x\|^2 - 2\alpha x_1 + \alpha^2 = 2(\|x\|^2 - \alpha x_1)$, using $\alpha^2 = \|x\|^2$. Hence $2v^T x/v^T v = 1$ and $Hx = x - v = \alpha e_1$. ■

The opposite sign choice, $\alpha = +\text{sign}(x_1)\|x\|$, also gives a reflection onto the axis. But then $v_1 = x_1 - \alpha$ subtracts two nearly equal numbers whenever x is nearly parallel to e_1 . This is the cancellation of Section 10.1, and it can make v inaccurate in every digit.

Theorem 10.3.2 — Householder QR

Let X be $n \times p$ with $n \geq p$.

- (a) There are reflections $H_1, \dots, H_p$, with H_j acting only on coordinates $j, \dots, n$ (or equal to I), such that

$$H_p \cdots H_1 X = \begin{pmatrix} R \\ 0 \end{pmatrix}$$

with R upper triangular of size $p \times p$. Thus $X = Q \begin{pmatrix} R \\ 0 \end{pmatrix}$ with $Q = H_1 \cdots H_p$ orthogonal, and $X = Q_1 R$ where Q_1 holds the first p columns of Q . If $\text{rank } X = p$, then R is nonsingular, and changing the signs of the rows of R with negative diagonal entries gives the Cholesky factor of $X^T X$.

- (b) The reduction takes $2np^2 - 2p^3/3$ flops to leading order. Applying $\mathbf{Q}^\top$ (or $\mathbf{Q}$) to a vector takes $4np - 2p^2$, and forming $\mathbf{Q}_1$ explicitly takes another $2np^2 - 2p^3/3$.
- (c) (Backward stability.) In floating-point arithmetic, the computed $\hat{\mathbf{R}}$ is the exact triangular factor of $\mathbf{X} + \Delta\mathbf{X}$, where each column satisfies $\|\Delta\mathbf{x}_j\| \leq c_{np}u\|\mathbf{x}_j\|$. The least squares solution $\hat{\mathbf{b}}$ computed from $\hat{\mathbf{R}}$ and the computed $\mathbf{Q}^\top\mathbf{y}$ is the exact least squares solution for data $(\mathbf{X} + \Delta\mathbf{X}, \mathbf{y} + \Delta\mathbf{y})$ with perturbations of the same kind, $\|\Delta\mathbf{y}\| \leq c_{np}u\|\mathbf{y}\|$. Here c_{np} is a modest multiple of np .

Proof.

- (a) Induction on the column. Suppose $\mathbf{H}_{j-1} \cdots \mathbf{H}_1\mathbf{X}$ has zeros below the diagonal in its first $j-1$ columns. Let $\mathbf{x}$ be the part of column j in rows $j, \dots, n$. If $\mathbf{x} = \mathbf{0}$, take $\mathbf{H}_j = \mathbf{I}$. Otherwise take the reflection of Lemma 10.3.1 for $\mathbf{x}$, acting on rows $j, \dots, n$. It zeroes column j below the diagonal. It leaves rows $1, \dots, j-1$ alone. It maps the first $j-1$ columns, which are zero in rows $j, \dots, n$, to themselves. After p steps the matrix is $\begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix}$, and $\mathbf{Q}$ is a product of orthogonal matrices. Orthogonal factors do not change rank, so $\text{rank } \mathbf{R} = \text{rank } \mathbf{X}$. If $\mathbf{D}$ is the diagonal matrix of signs of the diagonal of $\mathbf{R}$, then $\mathbf{X} = (\mathbf{Q}_1\mathbf{D})(\mathbf{D}\mathbf{R})$ is a QR factorization with positive diagonal, and $\mathbf{D}\mathbf{R}$ is the Cholesky factor by Theorem 10.2.1(d).
- (b) Step j builds $\mathbf{v}$ (about $2(n-j)$ flops) and applies the reflection to the $p-j$ remaining columns of length $n-j+1$, at about $4(n-j)(p-j)$ flops. Summing,

$$\sum_{j=1}^p 4(n-j)(p-j) \approx 4 \int_0^p (n-t)(p-t) dt = 2np^2 - \frac{2}{3}p^3.$$

Applying the p reflections to a vector costs $\sum_j 4(n-j+1) \approx 4np - 2p^2$. Forming $\mathbf{Q}_1$ means applying the reflections in reverse order to the first p columns of $\mathbf{I}_n$. Reflection j then acts on columns $j, \dots, p$ only, which gives the same integral again.

- (c) This is a theorem of rounding-error analysis, not of linear algebra, and we state it without proof. Its essence is that a reflection applied in floating point is an exact reflection applied to a slightly perturbed matrix, that orthogonal matrices do not amplify perturbations, and that the p steps add their perturbations. See Higham (2002, chapters 19 and 20) and Björck (1996, chapter 2).

■

Part (c) is why QR is the default. Its backward error is small relative to each column separately, which Section 10.5 uses to explain why column scaling does not change the accuracy of QR. Software never forms $\mathbf{Q}$; it stores the vectors $\mathbf{v}_j$ in the zeroed part of $\mathbf{X}$.

```
import numpy as np
import statsmodels.api as sm

def house(x):
    """Unit vector v with (I - 2 v v') x = alpha e_1, alpha = -sign(x_1) ||x||."""
    alpha = -np.copysign(np.linalg.norm(x), x[0])
    v = x.astype(float).copy()
    v[0] -= alpha # x_1 - alpha adds two numbers of the same
                  sign
    return v / np.linalg.norm(v), alpha
```

```

def householder_qr(X):
    """Return the reflection vectors and the p x p triangle R of X = Q [R; 0]."""
    A = np.array(X, dtype=float)
    n, p = A.shape
    vs = []
    for j in range(p):
        v, alpha = house(A[j:, j])
        A[j:, j:] -= 2.0 * np.outer(v, v @ A[j:, j:])    # apply I - 2vv' to the
        trailing block
        vs.append(v)
    return vs, np.triu(A[:p, :])

def apply_Qt(vs, y):
    """Q'y, applying the reflections H_1, ..., H_p in turn (Q itself is never formed)."""
    z = np.array(y, dtype=float)
    for j, v in enumerate(vs):
        z[j:] -= 2.0 * v * (v @ z[j:])
    return z

def apply_Q(vs, z):
    """Q z, applying the reflections in reverse order."""
    w = np.array(z, dtype=float)
    for j in range(len(vs) - 1, -1, -1):
        v = vs[j]
        w[j:] -= 2.0 * v * (v @ w[j:])
    return w

```

[Run this code online](#)

10.3.2 Regression quantities from the factorization

Section 6.10 derived the regression quantities from the thin factorization $\mathbf{X} = \mathbf{Q}_1 \mathbf{R}$ (Eq. 6.10.1). The full factorization, with $\mathbf{Q}$ square, adds one thing: the part of $\mathbf{Q}^\top \mathbf{y}$ that the model cannot reach, which is the residual in rotated coordinates.

Proposition 10.3.3 — Least squares from QR

Let $\mathbf{X} = \mathbf{Q} \begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix}$ with $\mathbf{Q}$ orthogonal and rank $\mathbf{X} = p$, and partition $\mathbf{c} = \mathbf{Q}^\top \mathbf{y}$ as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top)^\top$ with $\mathbf{c}_1 \in \mathbb{R}^p$. Then:

- (a) $\hat{\beta}$ is the solution of the triangular system $\mathbf{R}\mathbf{b} = \mathbf{c}_1$, and $\text{SSE} = \|\mathbf{c}_2\|^2$;
- (b) $\hat{\mathbf{Y}} = \mathbf{Q} \begin{pmatrix} \mathbf{c}_1 \\ \mathbf{0} \end{pmatrix}$ and $\hat{\boldsymbol{\varepsilon}} = \mathbf{Q} \begin{pmatrix} \mathbf{0} \\ \mathbf{c}_2 \end{pmatrix}$;
- (c) $(\mathbf{X}^\top \mathbf{X})^{-1} = \mathbf{R}^{-1} \mathbf{R}^{-\top}$ and $h_{ii} = \|\mathbf{e}_i^\top \mathbf{Q}_1\|^2$;
- (d) c_k^2 is the drop in residual sum of squares when column k is added to columns $1, \dots, k-1$, the sequential sum of squares of Definition 9.3.1.

Proof.

- (a) With $\mathbf{Q}_1$ the first p columns of $\mathbf{Q}$, $\mathbf{X} = \mathbf{Q}_1 \mathbf{R}$ is a thin QR factorization and $\mathbf{c}_1 = \mathbf{Q}_1^\top \mathbf{y}$, so $\mathbf{R}\hat{\beta} = \mathbf{c}_1$ and $\text{SSE} = \|\mathbf{y}\|^2 - \|\mathbf{c}_1\|^2$ are Eq. 6.10.1. What the full $\mathbf{Q}$

adds is $\|c\| = \|y\|$, hence $\text{SSE} = \|c_2\|^2$.

- (b) $X\hat{\beta} = Q \begin{pmatrix} R\hat{\beta} \\ 0 \end{pmatrix} = Q \begin{pmatrix} c_1 \\ 0 \end{pmatrix}$, and $y = Qc$.
- (c) $X^\top X = R^\top Q_1^\top Q_1 R = R^\top R$, and the leverage formula is Eq. 6.10.1.
- (d) The first k columns of X equal the first k columns of Q times the leading $k \times k$ block of R , which is nonsingular. So they span the same subspace as $q_1, \dots, q_k$, and the squared length of the projection of y onto it is $\sum_{j \leq k} (q_j^\top y)^2 = \sum_{j \leq k} c_j^2$ (Proposition 6.3.2).

■

Part (d) is the QR form of Proposition 10.2.2(b): by uniqueness of the triangle, c_1 equals the vector z of that proposition up to signs. The vector c_2 replaces the n correlated residuals by $n - p$ uncorrelated ones with the same sum of squares (Exercise (B1)).

Example. (Sequential sums of squares from one factorization)

For the state data of Example (Murder rates by the augmented factorization), Householder QR (Theorem 10.3.2) gives R with diagonal $-7.071, 21.511, 21.609$ and 113.413 . The first entry is $-\sqrt{50}$: the sign convention of Lemma 10.3.1 makes it negative. The squares of the first four entries of $Q^\top y$ are the sequential sums of squares: 1018.81 for the intercept (which is $n\bar{y}^2$), 102.49 for poverty, 80.04 for single-parent households given poverty, and 0.27 for urbanization given the other two. The remaining 46 entries have squared length $\text{SSE} = 101.96$. The urbanization variable adds almost nothing once the other two are in the model, which is what its small coefficient in Example (Poverty and murder rates) suggested. The script checks every number against nested fits, and checks the leverages against the squared row lengths of Q_1 .

```
data = sm.datasets.statelcrime.load_pandas().data.drop(index="District of Columbia")
y = data["murder"].to_numpy()
X = np.column_stack([np.ones(len(y)), data[["poverty", "single", "urban"]]])
n, p = X.shape

vs, R = householder_qr(X)
c = apply_Qt(vs, y)                # c = Q'y = (c1, c2)
c1, c2 = c[:p], c[p:]
beta = np.linalg.solve(R, c1)       # triangular system R b = c1
sse = c2 @ c2                       # SSE = ||c2||^2
resid = apply_Q(vs, np.concatenate([np.zeros(p), c2]))
seq_ss = c1 ** 2                    # sequential sums of squares, in column
    order
print("R diagonal:", np.round(np.diag(R), 4))
print("coefficients:", np.round(beta, 4), f" SSE = {sse:.4f}")
print("sequential SS (intercept, poverty, single, urban):", np.round(seq_ss, 3))
```

[Run this code online](#)

10.3.3 Givens rotations

A reflection zeroes a whole column at once. Sometimes it is better to zero one entry at a time, touching only two rows. This is the case when $\mathbf{X}$ is sparse, when rows arrive one by one, or when an existing factorization must be modified.

- Theorem 10.3.4 — Givens rotations** (a) For $(a, b) \neq (0, 0)$ let $r = (a^2 + b^2)^{1/2}$, $c = a/r$ and $s = b/r$. The rotation $\mathbf{G} = \begin{pmatrix} c & s \\ -s & c \end{pmatrix}$ is orthogonal and $\mathbf{G}(a, b)^\top = (r, 0)^\top$. Embedded in rows i and k of the identity, it changes only rows i and k of a matrix it multiplies, at 6 flops per column.
- (b) Zeroing the subdiagonal of $\mathbf{X}$ column by column, each entry against the row above it, produces $\mathbf{GX} = \begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix}$ with $\mathbf{G}$ orthogonal. It takes $np - p(p+1)/2$ rotations and $3np^2 - p^3$ flops to leading order.
- (c) (Row-wise QR.) Let $\mathbf{R}$ be upper triangular and $\mathbf{x} \in \mathbb{R}^p$. There are p rotations, the k th acting on row k of $\mathbf{R}$ and on the new row, that reduce $\begin{pmatrix} \mathbf{R} \\ \mathbf{x}^\top \end{pmatrix}$ to $\begin{pmatrix} \mathbf{R}_+ \\ \mathbf{0}^\top \end{pmatrix}$ with $\mathbf{R}_+$ upper triangular and $\mathbf{R}_+^\top \mathbf{R}_+ = \mathbf{R}^\top \mathbf{R} + \mathbf{x}\mathbf{x}^\top$. The cost is $3p^2$ flops. If $\mathbf{R}$ and $\mathbf{x}$ are augmented by the response, as in Proposition 10.2.2, the new row ends as $(0, \dots, 0, t)$, and the residual sum of squares increases by t^2 .
- (d) A rotation of rows i and k creates a nonzero in column j only if row i or row k already had one there.

Proof.

- (a) $c^2 + s^2 = 1$ gives $\mathbf{G}^\top \mathbf{G} = \mathbf{I}$. Also $ca + sb = (a^2 + b^2)/r = r$ and $-sa + cb = 0$. Each column of the product needs four multiplications and two additions.
- (b) When entry (i, j) is zeroed against row $i - 1$, working upwards from row n , both rows are already zero in columns $1, \dots, j - 1$, so no earlier zero is destroyed. Column j needs $n - j$ rotations, each acting on $p - j + 1$ columns, so the count is $\sum_j 6(n - j)(p - j + 1) \approx 6 \int_0^p (n - t)(p - t) dt = 3np^2 - p^3$.
- (c) Before rotation k , the new row is zero in columns $1, \dots, k - 1$, and so is row k of $\mathbf{R}$. The rotation chosen by (a) to zero the k th entry of the new row keeps both rows zero in those columns. After p rotations the new row is zero, and row k of $\mathbf{R}$ has been replaced by a row that is zero before column k . With $\mathbf{G}$ the product of the rotations,

$$\mathbf{R}_+^\top \mathbf{R}_+ = \begin{pmatrix} \mathbf{R} \\ \mathbf{x}^\top \end{pmatrix}^\top \mathbf{G}^\top \mathbf{G} \begin{pmatrix} \mathbf{R} \\ \mathbf{x}^\top \end{pmatrix} = \mathbf{R}^\top \mathbf{R} + \mathbf{x}\mathbf{x}^\top.$$

Rotation k acts on $p - k + 1$ columns, which gives $\sum_k 6(p - k + 1) \approx 3p^2$. In the augmented case, apply p rotations to the augmented $(p + 1) \times (p + 1)$ triangle $\mathbf{T}$ and the row $(\mathbf{x}^\top, y)$. The new row becomes $(\mathbf{0}^\top, t)$, and $\mathbf{T}_+^\top \mathbf{T}_+ = \mathbf{T}^\top \mathbf{T} + (\mathbf{x}^\top, y)^\top (\mathbf{x}^\top, y) - (\mathbf{0}^\top, t)^\top (\mathbf{0}^\top, t)$. The corner entry of $\mathbf{T}_+$ is still d , so the augmented triangle with the new row included is obtained by one more rotation, of d against t . This gives a corner $(d^2 + t^2)^{1/2}$, and the claim follows from Proposition 10.2.2(a).

- (d) Each new row is a combination of the two old ones. ■

Part (c) turns QR into a streaming algorithm (Gentleman 1973): feed the rows of $[\mathbf{X}, \mathbf{y}]$ one at a time into $\mathbf{R} = \mathbf{0}$. Memory is $O(p^2)$, one pass suffices, and the result is as accurate as Householder QR, so the one practical advantage of accumulating cross-products is kept without the κ^2 loss. Part (d) keeps $\mathbf{R}$ sparse for sparse designs, such as factors with many levels.

```
def givens(a, b):
    """c, s, r with [[c, s], [-s, c]] @ [a, b] = [r, 0]."""
    if b == 0.0:
        return 1.0, 0.0, a
    r = np.hypot(a, b)                # sqrt(a^2 + b^2) without overflow
    return a / r, b / r, r
```

[Run this code online](#)

```
def add_row(R, z, x, y):
    """Rotate one observation (x, y) into the triangle R and the vector z = (Q'y)[:p].

    Returns the new R, z and the entry that leaves: its square is the increase in SSE.
    """
    R, z, x, y = R.copy(), z.copy(), np.array(x, dtype=float), float(y)
    for k in range(len(x)):
        c, s, r = givens(R[k, k], x[k])
        Rk, xk = R[k, k:].copy(), x[k:].copy()
        R[k, k:], x[k:] = c * Rk + s * xk, -s * Rk + c * xk
        z[k], y = c * z[k] + s * y, -s * z[k] + c * y
    return R, z, y

def rowwise_ls(X, y):
    """Least squares with one pass over the rows and O(p^2) memory."""
    p = X.shape[1]
    R, z, sse = np.zeros((p, p)), np.zeros(p), 0.0
    for xi, yi in zip(X, y):
        R, z, leftover = add_row(R, z, xi, yi)
        sse += leftover ** 2
    return R, z, sse

R_row, z_row, sse_row = rowwise_ls(X, y)
print("coefficients (row-wise Givens):", np.round(np.linalg.solve(R_row, z_row), 4))
print(f"SSE (row-wise Givens): {sse_row:.4f}")
```

[Run this code online](#)

The rotation in `givens` uses `hypot`, which computes $(a^2 + b^2)^{1/2}$ without overflow or underflow. Square-root-free “fast” variants (Gentleman 1973) matter less on modern hardware.

10.3.4 Gram–Schmidt revisited

Gram–Schmidt (Theorem 1.5.2) also computes a QR factorization, at $2np^2$ flops, and forms $\mathbf{Q}_1$ explicitly, which gives leverages directly. The classical form computes $r_{kj} = \mathbf{q}_k^\top \mathbf{x}_j$ from the *original* column; the modified form subtracts each new direction from all remaining columns at once. The two agree in exact arithmetic and differ greatly in floating point.

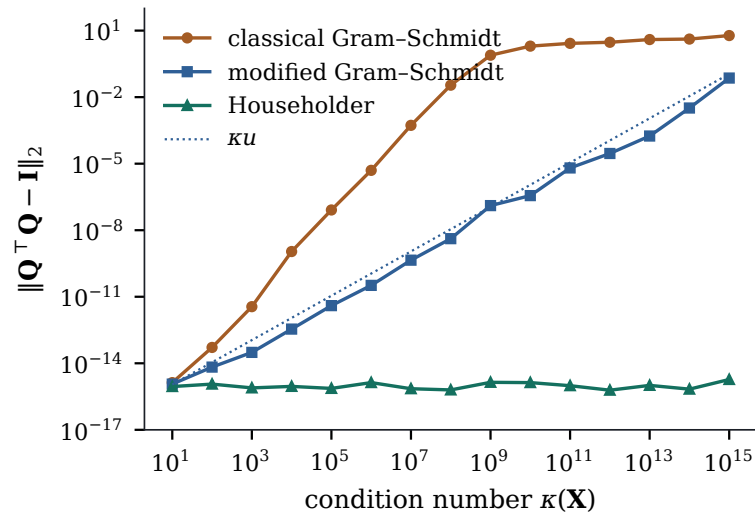


Figure 10.3.1: Departure from orthogonality of the computed $\mathbf{Q}$ against the condition number of $\mathbf{X}$, for classical and modified Gram–Schmidt and for Householder QR. The dotted line is κu .

Example. (Loss of orthogonality)

For 100×12 matrices with prescribed condition numbers from 10 to 10^{15} , Figure 10.3.1 plots $\|\hat{\mathbf{Q}}^T \hat{\mathbf{Q}} - \mathbf{I}\|_2$ for the computed $\hat{\mathbf{Q}}$. At $\kappa = 10^8$ classical Gram–Schmidt gives 3.5×10^{-2} , and its columns are no longer an orthonormal basis in any useful sense. Modified Gram–Schmidt gives 4.2×10^{-9} , in line with the bound $O(\kappa u)$ of Björck (1967). The $\mathbf{Q}$ accumulated from Householder reflections gives 6.3×10^{-16} , orthogonal to working precision whatever the condition number.

```
def cgs(X):
    """Classical Gram-Schmidt: inner products with the original column."""
    n, p = X.shape
    Q = np.zeros((n, p))
    for j in range(p):
        w = X[:, j] - Q[:, :j] @ (Q[:, :j].T @ X[:, j])
        Q[:, j] = w / np.linalg.norm(w)
    return Q

def mgs(X):
    """Modified Gram-Schmidt: subtract each direction from the working vector at once."""
    Q = np.array(X, dtype=float)
    n, p = Q.shape
    for j in range(p):
        Q[:, j] /= np.linalg.norm(Q[:, j])
        Q[:, j + 1:] -= np.outer(Q[:, j], Q[:, j] @ Q[:, j + 1:])
    return Q
```

[Run this code online](#)

Loss of orthogonality does not make modified Gram–Schmidt useless. It is numerically equivalent to Householder QR applied to $\mathbf{X}$ with p rows of zeros on top (Björck and Paige 1992), so applied to $[\mathbf{X}, \mathbf{y}]$, with $\mathbf{Q}_1^T \mathbf{y}$ taken from the last column, it gives a backward stable least squares solution. Leverages computed from the rows of $\hat{\mathbf{Q}}_1$ are not reliable when κ is large.

Idea. Orthogonal transformations reduce $\mathbf{X}$ to a triangle while preserving every residual length. Householder reflections do it fastest; Givens rotations do it one entry at a time, which suits streaming, updating and sparse data.

10.3.5 Exercises

10.3.5.1 A. Check your understanding

Exercise A1.

Find the Householder vector and α of [Lemma 10.3.1](#) for $\mathbf{x} = (3, 4)^\top$, and verify $\mathbf{H}\mathbf{x} = \alpha\mathbf{e}_1$. For $\mathbf{x} = (1, 10^{-9})^\top$, compute v_1 with both sign choices in double precision, and explain what goes wrong with the unstable one.

Solution. $\|\mathbf{x}\| = 5$, $\alpha = -5$, $\mathbf{v} = (8, 4)^\top$, $\mathbf{v}^\top\mathbf{v} = 80$ and $\mathbf{v}^\top\mathbf{x} = 40$, so $\mathbf{H}\mathbf{x} = \mathbf{x} - 2 \cdot \frac{40}{80}\mathbf{v} = (3, 4) - (8, 4) = (-5, 0)$. For $\mathbf{x} = (1, 10^{-9})^\top$, $\|\mathbf{x}\| = (1 + 10^{-18})^{1/2}$ rounds to 1. The stable choice gives $v_1 = 1 + 1 = 2$. The unstable one gives $v_1 = 1 - 1 = 0$ in floating point, while the true value is $1 - \|\mathbf{x}\| \approx -5 \times 10^{-19}$. With $\mathbf{v} = (0, 10^{-9})$ the “reflection” swaps the sign of the second coordinate and maps $\mathbf{x}$ to $(1, -10^{-9})$, not onto the axis. An algebraically equivalent formula, $v_1 = -(x_2^2 + \cdots + x_m^2)/(x_1 + \|\mathbf{x}\|)$, rescues that choice.

10.3.5.2 B. Practice

Exercise B1.

In the setting of [Proposition 10.3.3](#), let $\mathbf{Q} = [\mathbf{Q}_1, \mathbf{Q}_2]$. Show that under the linear model with $\text{Cov}(\boldsymbol{\varepsilon}) = \sigma^2\mathbf{I}$, the vector $\mathbf{c}_2 = \mathbf{Q}_2^\top\mathbf{y}$ has mean $\mathbf{0}$ and covariance $\sigma^2\mathbf{I}_{n-p}$, and that $\hat{\boldsymbol{\varepsilon}} = \mathbf{Q}_2\mathbf{c}_2$. Under normality, deduce from $\mathbf{c}_2$ alone that $\text{SSE}/\sigma^2 \sim \chi^2(n-p)$ independently of $\hat{\boldsymbol{\beta}}$ (compare Theorem 7.5.1). Is $\mathbf{c}_2$ unique?

Solution. $\mathbf{Q}_2^\top\mathbf{X} = \mathbf{0}$, because $\mathbf{Q}^\top\mathbf{X} = \begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix}$. So $\mathbf{c}_2 = \mathbf{Q}_2^\top\mathbf{X}\boldsymbol{\beta} + \mathbf{Q}_2^\top\boldsymbol{\varepsilon} = \mathbf{Q}_2^\top\boldsymbol{\varepsilon}$, with mean $\mathbf{0}$ and covariance $\sigma^2\mathbf{Q}_2^\top\mathbf{Q}_2 = \sigma^2\mathbf{I}$. By [Proposition 10.3.3\(b\)](#), $\hat{\boldsymbol{\varepsilon}} = \mathbf{Q}\begin{pmatrix} \mathbf{0} \\ \mathbf{c}_2 \end{pmatrix} = \mathbf{Q}_2\mathbf{c}_2$. Under normality, $\mathbf{Q}^\top\boldsymbol{\varepsilon} \sim N_n(\mathbf{0}, \sigma^2\mathbf{I})$ (Theorem 3.3.1), so $\mathbf{c}_1 - \mathbf{R}\boldsymbol{\beta} = \mathbf{Q}_1^\top\boldsymbol{\varepsilon}$ and $\mathbf{c}_2 = \mathbf{Q}_2^\top\boldsymbol{\varepsilon}$ are independent normal vectors with identity covariance times σ^2 . Then $\text{SSE}/\sigma^2 = \|\mathbf{c}_2/\sigma\|^2 \sim \chi^2(n-p)$, and $\hat{\boldsymbol{\beta}} = \mathbf{R}^{-1}\mathbf{c}_1$ is a function of $\mathbf{c}_1$ alone. $\mathbf{c}_2$ is not unique: $\mathbf{Q}_2$ can be replaced by $\mathbf{Q}_2\mathbf{O}$ for any orthogonal $\mathbf{O}$, which changes $\mathbf{c}_2$ but not its length or its distribution.

Exercise B2.

Weighted least squares with a diagonal weight matrix $\mathbf{W}$ of positive weights is ordinary least squares for $\mathbf{W}^{1/2}\mathbf{X}$ and $\mathbf{W}^{1/2}\mathbf{y}$. Show that $\kappa(\mathbf{W}^{1/2}\mathbf{X}) \leq \kappa(\mathbf{W}^{1/2})\kappa(\mathbf{X})$. Give an example with two very different weights in which $\kappa(\mathbf{W}^{1/2}\mathbf{X})$ is large although $\kappa(\mathbf{X})$ is small, and one in which it is small although $\kappa(\mathbf{W}^{1/2})$ is large.

Solution. $\sigma_{\max}(\mathbf{W}^{1/2}\mathbf{X}) \leq \sigma_{\max}(\mathbf{W}^{1/2})\sigma_{\max}(\mathbf{X})$ and $\|\mathbf{W}^{1/2}\mathbf{X}\mathbf{v}\| \geq \sigma_{\min}(\mathbf{W}^{1/2})\|\mathbf{X}\mathbf{v}\| \geq \sigma_{\min}(\mathbf{W}^{1/2})\sigma_{\min}(\mathbf{X})$ for unit $\mathbf{v}$, by Proposition 1.8.2. Divide. For the first example take

$\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ (with $\kappa = 1$) and $\mathbf{W} = \text{diag}(1, w^2)$ with w large. Then $\mathbf{W}^{1/2}\mathbf{X}$ has rows $(1, 1)$ and $(w, -w)$, whose singular values are $\sqrt{2}$ and $\sqrt{2}w$, so $\kappa = w$. For the second take $\mathbf{X} = \begin{pmatrix} 1 & 0 \\ 0 & 1/w \end{pmatrix}$ with the same $\mathbf{W}$. Then $\mathbf{W}^{1/2}\mathbf{X} = \mathbf{I}$.

10.3.5.3 C. Going deeper

Exercise C1.

Show that, in exact arithmetic, Householder QR ([Theorem 10.3.2](#)) applied to $\begin{pmatrix} \mathbf{0}_{p \times p} \\ \mathbf{X} \end{pmatrix}$ produces the same $\mathbf{R}$ as modified Gram–Schmidt applied to $\mathbf{X}$ up to the signs of its rows, and that with the sign convention of [Lemma 10.3.1](#) its reflection vectors are $\mathbf{v}_j = \begin{pmatrix} \mathbf{e}_j \\ \mathbf{q}_j \end{pmatrix} / \sqrt{2}$. This is the observation of Björck and Paige (1992), who take $\alpha = +\|\mathbf{x}\|$ and so obtain $\begin{pmatrix} -\mathbf{e}_j \\ \mathbf{q}_j \end{pmatrix} / \sqrt{2}$.